

HabotConnect Hiring Project

Candidate Name: Amruta Suryavanshi

Email: amrutasuryavanshi212@gmail.com

Phone: 9503700849

Table of Contents

1. Project Overview
2. Project Objectives
3. Technology Stack
4. Architecture
5. Project Folder Structure
6. Task 1
7. Task 2
8. Task 3
9. Security
10. Testing
11. Challenges
12. Future Improvements
13. Conclusion
14. References

Project Overview

This project simulates a real-world production scenario where security vulnerabilities and data validation issues must be resolved using DevOps best practices. It demonstrates secure Google Cloud infrastructure provisioning with Terraform, automated CI/CD build gates, and robust data validation using Django REST Framework to ensure secure, reliable, and compliant application deployments.

Project Objectives

- Provision secure GCP resources using Terraform.
- Implement IAM, RBAC, and Row-Level Security (RLS).
- Build a fail-closed CI/CD pipeline with automated linting and security scanning.
- Detect and block hardcoded secrets in code.
- Validate student onboarding data using Django REST Framework serializers.
- Ensure data integrity and schema consistency for analytics.
- Demonstrate DevOps automation, cloud security, and Infrastructure as Code (IaC) best practices.

Technologies Used

Cloud Platform

- Google Cloud Platform (GCP)

Infrastructure as Code (IaC)

- Terraform

Cloud Services

- Google Cloud Storage (GCS)
- BigQuery
- Identity and Access Management (IAM)

Backend

- Python 3
- Django
- Django REST Framework (DRF)

CI/CD

- GitHub Actions

Code Quality & Security

- Black (Python Code Formatter)
- Flake8 (Python Linter)
- Bandit (Security Scanner)
- Detect-Secrets (Secret Detection)
- Terraform fmt
- Terraform validate
- Terraform plan

Version Control

- Git
- GitHub

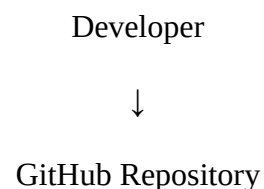
Development Tools

- Visual Studio Code
- Google Cloud CLI (gcloud)
- Terraform CLI

Architecture

The project follows a secure DevOps workflow that automates infrastructure provisioning, code quality checks, security validation, and data validation before deployment.

Architecture Flow





GitHub Actions (CI Pipeline)



Source Code Checkout



Install Dependencies



Code Formatting (Black)



Python Linting (Flake8)



Security Scan (Bandit)



Secret Detection (Detect-Secrets)



Terraform Format Check



Terraform Validation



Terraform Plan



Pipeline Status

- If any validation fails → Build is blocked (Fail-Closed)
- If all validations pass → Infrastructure deployment is approved



Terraform



Google Cloud Platform

- Google Cloud Storage (D0 Raw Landing Bucket)
- BigQuery Dataset (D1 Staged/Enforced)
- IAM Role Assignments (Least Privilege)



Django REST Framework API



Serializer Validation (DCYN Logic)



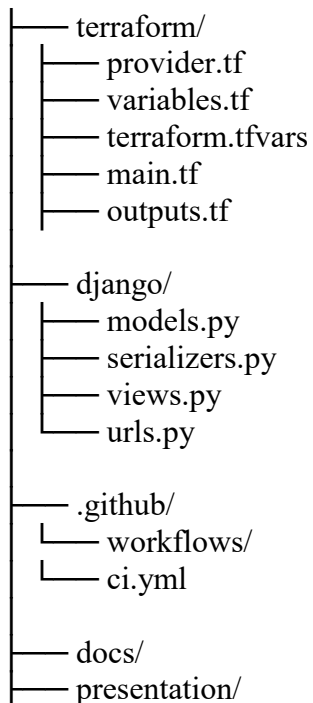
Validated Data



BigQuery Analytics

Folder Structure

habot-devops-project/



Task 1 – Terraform Infrastructure Provisioning

1. Infrastructure as Code (IaC)

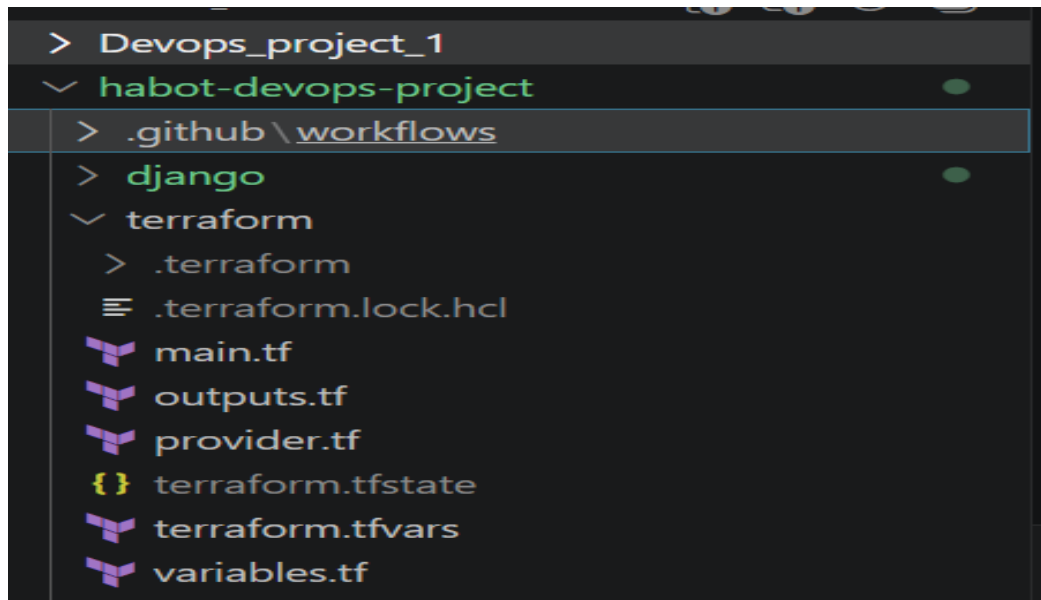
- Infrastructure is created using code instead of manual configuration.

- Terraform automates cloud resource provisioning.
- Ensures consistent and repeatable deployments.
- Reduces human errors.
- Infrastructure can be version controlled using Git.

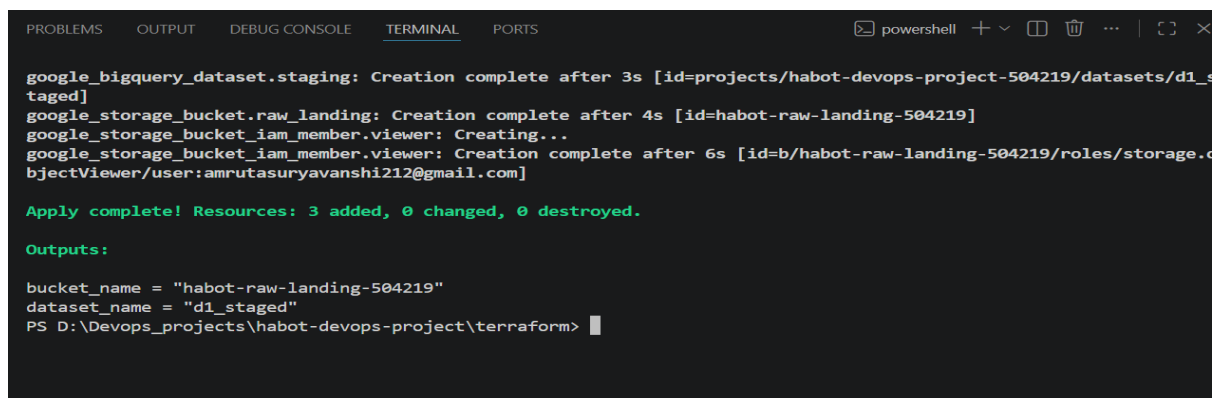
Code: provider.tf, main.tf, variables.tf

Screenshot :

- Terraform project structure



- terraform apply output



2. Terraform Provider

- Connects Terraform to Google Cloud Platform.
- Authenticates using Google Cloud credentials.
- Specifies the target project and region.
- Allows Terraform to create and manage GCP resources.

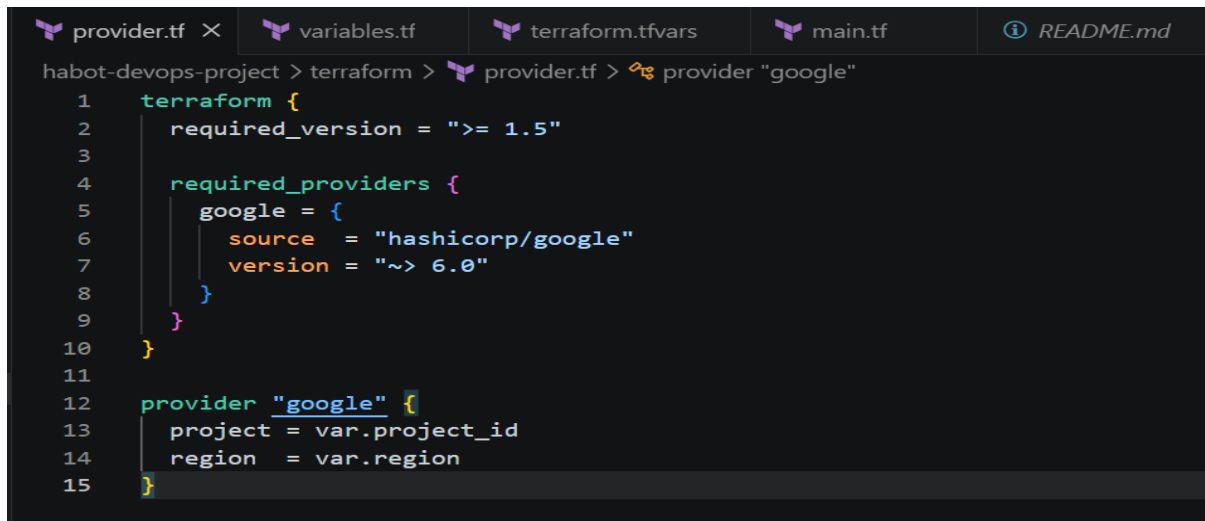
Code: provider.tf

```
provider "google" {
```

```
project = var.project_id
region = var.region
}
```

Screenshot :

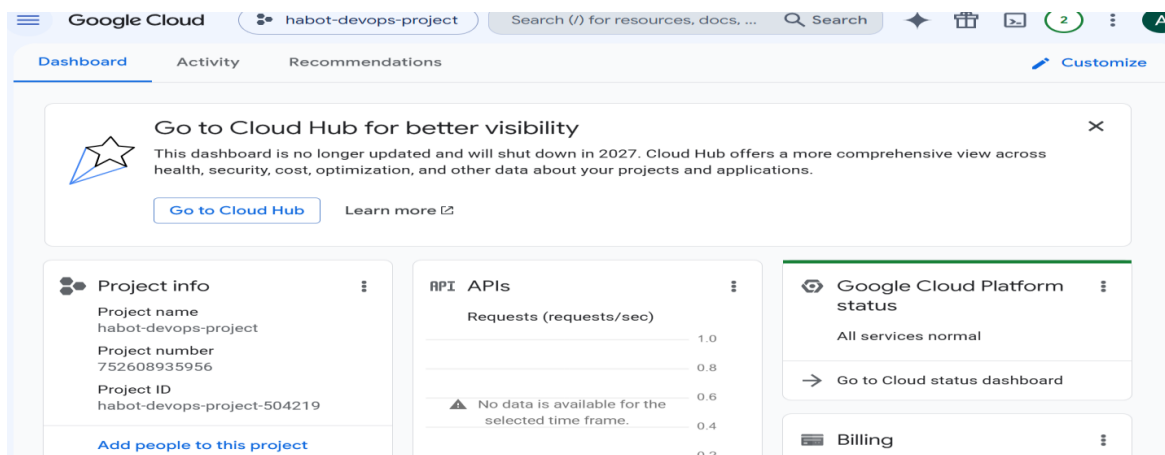
- Provider configuration



The screenshot shows a Terraform configuration file named `provider.tf` within a project named `habot-devops-project`. The configuration defines the Google Cloud provider with the following details:

- `terraform {`
 - `required_version = ">= 1.5"`
 - `required_providers {`
 - `google = {`
 - `source = "hashicorp/google"`
 - `version = "~> 6.0"`
- `provider "google" {`
 - `project = var.project_id`
 - `region = var.region`

- GCP project details



3. Variables

- Store reusable configuration values.
- Avoid hardcoding project-specific information.
- Improve code readability and maintainability.
- Easily deploy the same code to different environments.

Examples:

- Project ID
- Region
- Bucket Name
- Dataset Name

Code: variables.tf, terraform.tfvars

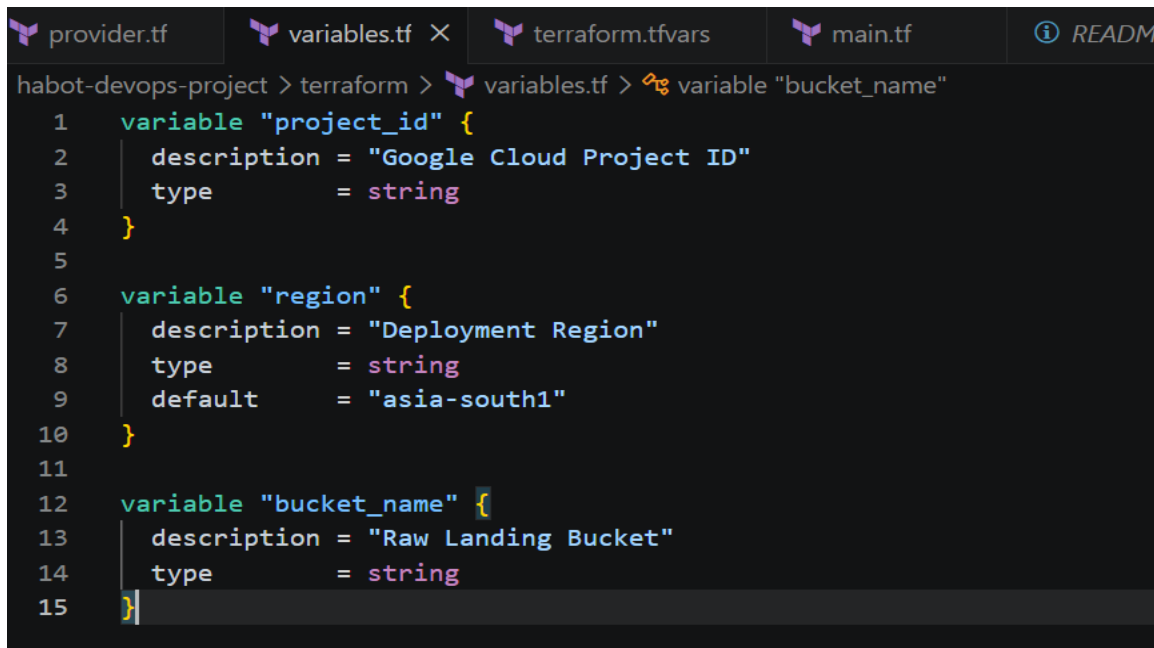
```
variable "project_id" {
  description = "Google Cloud Project ID"
  type       = string
}

variable "region" {
  description = "Deployment Region"
  type       = string
  default    = "asia-south1"
}

variable "bucket_name" {
  description = "Raw Landing Bucket"
  type       = string
}
```

Screenshot:

- Variables file



```
habot-devops-project > terraform > variables.tf > variable "bucket_name"

1  variable "project_id" {
2    description = "Google Cloud Project ID"
3    type       = string
4  }
5
6  variable "region" {
7    description = "Deployment Region"
8    type       = string
9    default    = "asia-south1"
10 }
11
12 variable "bucket_name" {
13   description = "Raw Landing Bucket"
14   type       = string
15 }
```

4. Google Cloud Storage Bucket

- Created as the **Raw Landing (D0)** storage.
- Stores incoming student onboarding data.
- Configured with secure settings.
- Supports future data processing.

Security Features:

- Uniform Bucket-Level Access
- Public Access Prevention
- Versioning

Code: main.tf

```
resource "google_storage_bucket" "raw_landing" {
  name     = var.bucket_name
  location = "ASIA"

  uniform_bucket_level_access = true

  versioning {
    enabled = true
  }

  public_access_prevention = "enforced"

  force_destroy = false
}
```

Screenshot:

- Bucket overview

Buckets

+ Create

Refresh

Go to path

Learn

Filter

Filter buckets

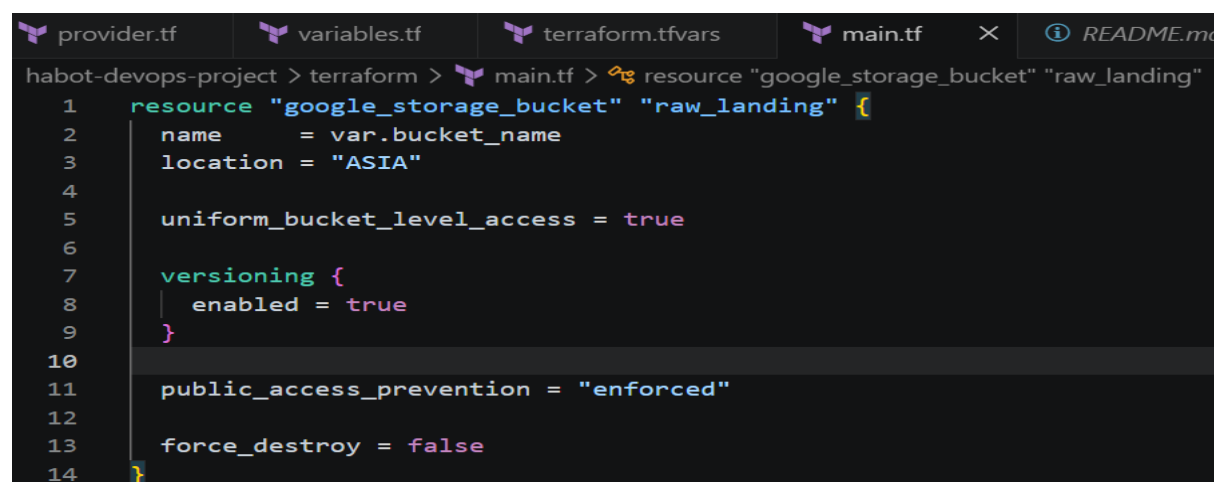
?

⚙

☰

☐ Name ↑	Created	Location type	Location	Default storage class ?	Last modified
☐ habot-raw-landing-504219	Aug 2, 2026, 1:41:59 AM	Multi-region	asia	Standard	Aug 2, 2026, 1:41:59 AM

- Bucket configuration



```
provider.tf  variables.tf  terraform.tfvars  main.tf  README.md
habot-devops-project > terraform > main.tf > resource "google_storage_bucket" "raw_landing"
1  resource "google_storage_bucket" "raw_landing" {
2      name     = var.bucket_name
3      location = "ASIA"
4
5      uniform_bucket_level_access = true
6
7      versioning {
8          enabled = true
9      }
10
11     public_access_prevention = "enforced"
12
13     force_destroy = false
14 }
```

5. BigQuery Dataset

- Created as the **Staged (D1)** data layer.
- Stores validated and structured data.
- Used for reporting and analytics.
- Integrates with future data pipelines.

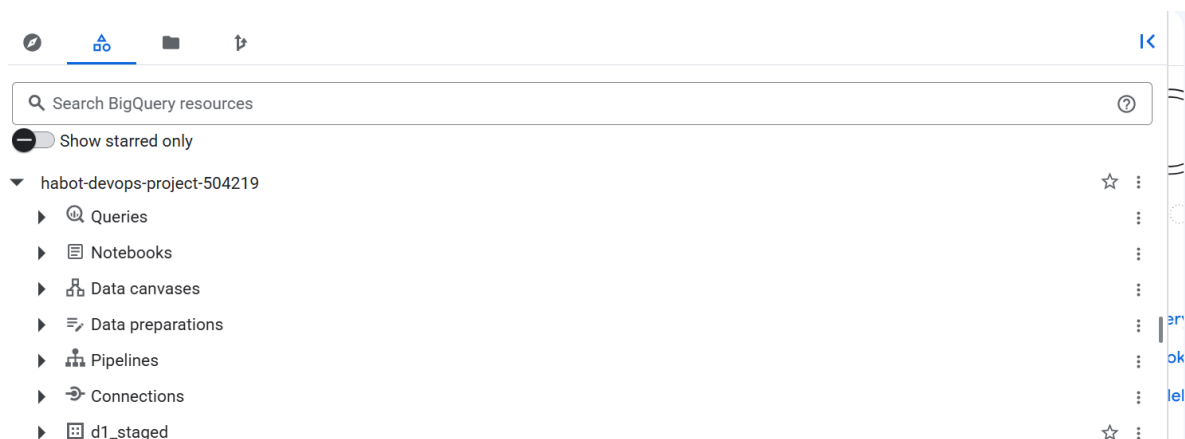
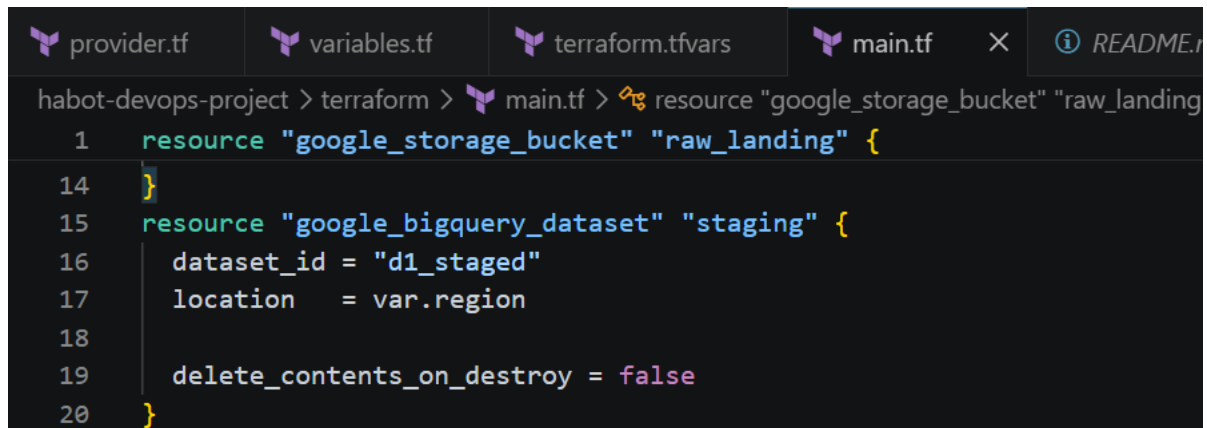
Code: main.tf

```
resource "google_bigquery_dataset" "staging" {
  dataset_id = "d1_staged"
  location   = var.region

  delete_contents_on_destroy = false
}
```

Screenshot:

- BigQuery dataset



6. BigQuery Row-Level Security (RLS)

Purpose

- Restricts access to specific rows in the BigQuery table.
- Protects sensitive student information.
- Ensures users only view authorized records.
- Supports secure analytics and reporting.

Implementation

- Applied on the 'student_data' table.
- Uses a BigQuery Row Access Policy.
- Filters rows based on the 'school' column.
- Configured using SQL DDL after Terraform deployment.

SQL

```
CREATE ROW ACCESS POLICY school_policy
ON `habot-devops-project-504219.d1_staged.student_data`
GRANT TO ("user:amrutasuryavanshi212@gmail.com")
FILTER USING (school = 'ABC School');
```

Benefits

- Protects confidential student data.
- Implements the Principle of Least Privilege.
- Prevents unauthorized access to table rows.
- Improves data privacy and security.

Screenshot

- Row Access Policy page

Row access policies ✕

Limit access for a group of users to a subset of table rows. To add, edit, or remove row access policies, use DDL statements. [Learn more](#)

Name	Filter expression	Created	Last modified	
school_policy	school = 'ABC School'	Aug 2, 2026, 9:25:52 PM	Aug 2, 2026, 9:25:52 PM	View

- BigQuery table with RLS configured

student_data x *Untitled query x

habot-devops-project-504219 / Datasets / d1_staged / Tables / student_data

student_data Query Open in Share

Schema Details Preview Table Explorer Preview Insights Lineage Data Profile Data

Field name	Type	Mode	Description	Key	Collation	Default Value	Display name
student_name	STRING	REQUIRED	-	-	-	-	-
school	STRING	REQUIRED	-	-	-	-	-
city	STRING	NULLABLE	-	-	-	-	-
age	INTEGER	NULLABLE	-	-	-	-	-

Untitled query Run Save Schedule Share

```
1 CREATE ROW ACCESS POLICY school_policy
2 ON `habot-devops-project-504219.d1_staged.student_data`
3 GRANT TO ("user:amrutasuryavanshi212@gmail.com")
4 FILTER USING (school = 'ABC School');
```

7. Identity and Access Management (IAM)

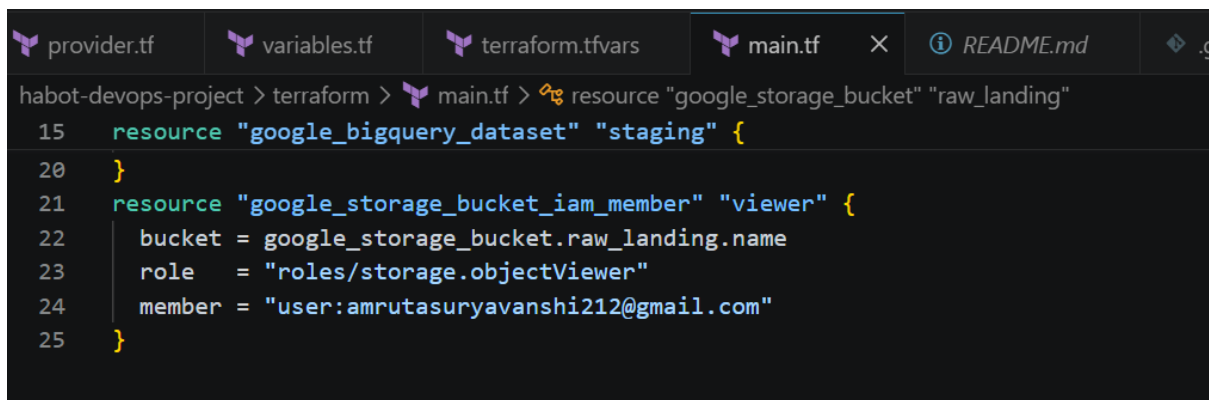
- Controls access to cloud resources.
- Assigns roles to users and service accounts.
- Prevents unauthorized access.
- Implements Role-Based Access Control (RBAC).

Code: main.tf

```
resource "google_storage_bucket_iam_member" "viewer" {  
  bucket = google_storage_bucket.raw_landing.name  
  role   = "roles/storage.objectViewer"  
  member = "user:amrutasuryavanshi212@gmail.com"  
}
```

Screenshot:

- IAM permissions



The screenshot shows a code editor with several tabs: provider.tf, variables.tf, terraform.tfvars, main.tf, and README.md. The main.tf tab is active, showing the following Terraform configuration:

```
habot-devops-project > terraform > main.tf > resource "google_storage_bucket" "raw_landing"  
15 resource "google_bigquery_dataset" "staging" {  
20 }  
21 resource "google_storage_bucket_iam_member" "viewer" {  
22     bucket = google_storage_bucket.raw_landing.name  
23     role   = "roles/storage.objectViewer"  
24     member = "user:amrutasuryavanshi212@gmail.com"  
25 }
```

Permissions

[View by principals](#)

[View by roles](#)

[Grant access](#)

[Remove access](#)

Filter Enter property name or value

☐ Type	Principal ↑	Name	Role	
☐	amrutasuryavanshi212@gmail.com	Amruta Suryavanshi	Storage Object Viewer	

8. Principle of Least Privilege

- Grants only the minimum permissions required.
- Prevents unnecessary administrative access.
- Improves cloud security.
- Reduces the risk of accidental changes.

Screenshot:

- IAM role assignment

Google Cloud habot-devops-project Search (/) for resources, docs, products, and more

Bucket details Edit bucket Go to path Refr

Not public

This bucket is not publicly accessible since public access is being prevented. Because of this restriction, objects cannot be publicly shared over the internet. [Learn more](#)

Principals restricted from bucket access:
allUsers, allAuthenticatedUsers

[Remove Public Access Prevention](#)

Uniform: No object-level ACLs enabled

90 days left to change this setting

All object access is controlled by bucket permissions and objects cannot have their own access control lists (ACLs). To allow per-object access, you can switch to fine-grained access within 90 days. [Learn more](#)

[Switch to fine-grained](#)

Permissions

View by principals View by roles

[Grant access](#) [Remove access](#)

Filter Enter property name or value

Type	Principal ↑	Name	Role
	amrutasuryavanshi212@gmail.com	Amruta Suryavanshi	Storage Object Viewer

9. Public Access Prevention

- Blocks public access to the Cloud Storage bucket.
- Ensures only authorized users can access data.
- Protects sensitive student information.
- Enabled at the bucket level.

Verification:

Cloud Storage → Bucket → Protection → Public Access Prevention = **Enforced**

Screenshot:

- Public Access Prevention enabled

Google Cloud habot-devops-project Search (/) for resources, docs, products, and more

Bucket details Edit bucket Go to path Refresh

Replication Default

Cross-bucket replication Not enabled

Default storage class Standard

Requester Pays Off

Tags None

Labels goog-terraform-provisioned : true

Cloud Console URL <https://console.cloud.google.com/storage/browser/habot-raw-landing-504219>

Cloud Storage URI <gs://habot-raw-landing-504219>

Cross-origin resource sharing Not enabled

Permissions

Access control Uniform

Public access prevention Enabled via bucket setting

Public access status Not public

IP filtering Not configured

10. Versioning

- Keeps previous versions of uploaded files.
- Protects against accidental deletion or overwriting.

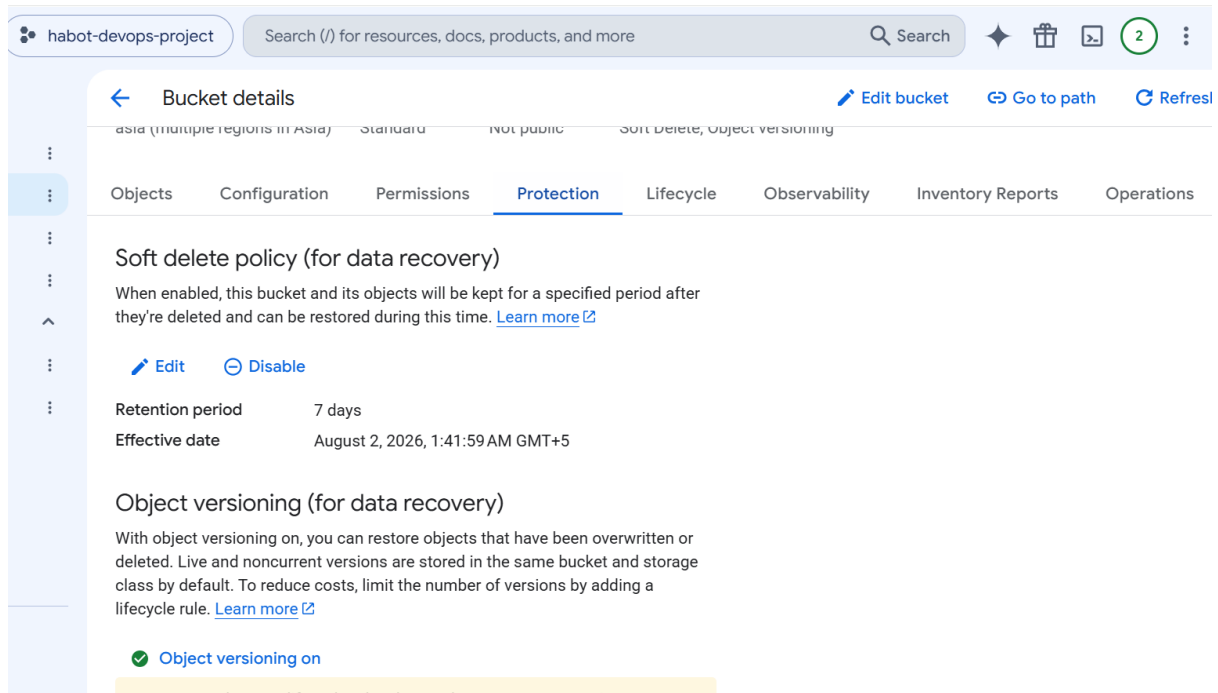
- Allows recovery of older file versions.
- Improves data reliability.

Verification:

Cloud Storage → Bucket → Protection → Versioning = **Enabled**

Screenshot:

- Versioning enabled



11. Terraform Deployment Steps

1. Write Terraform configuration files.
2. Run terraform init.

```
PS D:\Devops_projects\habot-devops-project\terraform> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/google from the dependency lock file
- Using previously-installed hashicorp/google v6.50.0

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/google
- Using previously-installed hashicorp/google v6.50.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS D:\Devops_projects\habot-devops-project\terraform>
```

3. Run terraform fmt.
4. Run terraform validate.

```
PS D:\Devops_projects\habot-devops-project\terraform> terraform fmt
PS D:\Devops_projects\habot-devops-project\terraform> terraform validate
Success! The configuration is valid.

PS D:\Devops_projects\habot-devops-project\terraform> 
```

5. Run terraform plan.

```
PS D:\Devops_projects\habot-devops-project\terraform> terraform plan
google_bigquery_dataset.staging: Refreshing state... [id=projects/habot-devops-project-504219/datasets/d1_staged]
google_storage_bucket.raw_landing: Refreshing state... [id=habot-raw-landing-504219]
google_storage_bucket_iam_member.viewer: Refreshing state... [id=b/habot-raw-landing-504219/roles/storage.objectV
r:amrutasuryavanshi212@gmail.com]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no change
needed.
PS D:\Devops_projects\habot-devops-project\terraform> 
```

6. Run terraform apply.

```
google_bigquery_dataset.staging: Creation complete after 3s [id=projects/habot-devops-project-504219/datasets/d1_s
tagged]
google_storage_bucket.raw_landing: Creation complete after 4s [id=habot-raw-landing-504219]
google_storage_bucket_iam_member.viewer: Creating...
google_storage_bucket_iam_member.viewer: Creation complete after 6s [id=b/habot-raw-landing-504219/roles/storage.o
bjectViewer/user:amrutasuryavanshi212@gmail.com]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

bucket_name = "habot-raw-landing-504219"
dataset_name = "d1_staged"
PS D:\Devops_projects\habot-devops-project\terraform> 
```

7. Verify resources in Google Cloud Console.

The screenshot shows the Google Cloud Console interface for the project 'habot-devops-project'. At the top, there's a navigation bar with the Google Cloud logo, the project name, and a search bar. Below this, there's a 'Dashboard' tab selected, with 'Activity' and 'Recommendations' also visible. A prominent banner at the top of the dashboard area encourages users to 'Go to Cloud Hub for better visibility', stating that the current dashboard is no longer updated and will be shut down in 2027. Below the banner, the dashboard is organized into several sections: 'Project info' on the left, 'API APIs' in the center, and 'Google Cloud Platform status' and 'Billing' on the right. The 'Project info' section lists the project name 'habot-devops-project', project number '752608935956', and project ID 'habot-devops-project-504219'. The 'API APIs' section shows a line graph for 'Requests (requests/sec)' with a scale from 0.2 to 1.0, and a message indicating 'No data is available for the selected time frame.' The 'Google Cloud Platform status' section shows 'All services normal' and a link to 'Go to Cloud status dashboard'. The 'Billing' section is partially visible at the bottom right.

Task 2 – Poka-Yoke Automated CI/CD Build Gate

Task 2 Overview

The objective of Task 2 is to create an automated **CI/CD pipeline** using **GitHub Actions**. The pipeline automatically checks code quality, security, and Terraform configuration before allowing deployment. If any validation fails, the pipeline immediately stops (**Fail-Closed**) to prevent insecure or incorrect code from being deployed.

1. Why CI/CD?

- Automates code validation after every push.
- Detects errors early in the development process.
- Maintains code quality and consistency.
- Reduces manual testing effort.
- Prevents deployment of faulty code.

Purpose: Improve software quality through automation.

2. Why Fail-Closed?

- Stops the pipeline immediately when an error is detected.
- Prevents insecure or invalid code from being deployed.
- Ensures only validated code reaches the next stage.
- Increases application security and reliability.

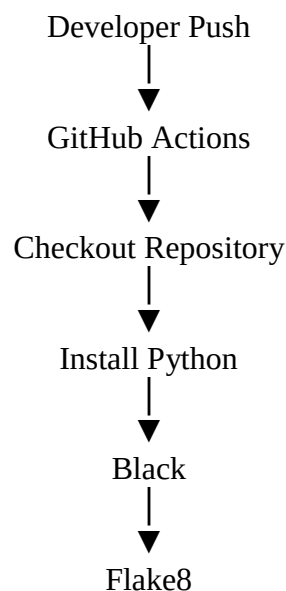
Purpose: Block deployment if any validation fails.

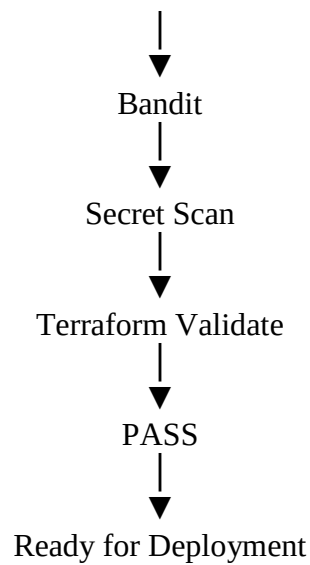
3. Poka-Yoke (Mistake-Proofing)

- Automates checks to eliminate human errors.
- Prevents accidental commits of bad or insecure code.
- Enforces coding standards automatically.
- Ensures every commit follows project rules.

Purpose: Prevent mistakes before deployment.

4. CI/CD Workflow





5. Testing Fail-Closed Pipeline

Test 1 – Black

- Remove proper spacing or formatting.
- **Expected:** Black fails and pipeline stops.

The screenshot shows a GitHub Actions workflow run for the repository `github.com/SuryavanshiAmruta/habot-devops-project`. The workflow is named `security-checks` and failed 1 minute ago. The failure is in the `Check Formatting` step, which is part of the `security-checks` job. The logs show that the `black` command was run to check for formatting issues. It found 4 files that would be reformatted: `django/urls.py`, `django/views.py`, `django/madels.py`, and `django/serializers.py`. The process completed with exit code 1, indicating a failure.

1 error and 1 warning

Summary

All jobs

security-checks

Run details

Usage

Workflow file

security-checks
failed 1 minute ago in 13s

Search logs

Set up job 1s

Checkout Repository 1s

Setup Python 0s

Install Python Tools 7s

Check Formatting 1s

```
1 ▶ Run black --check .
11 would reformat /home/runner/work/habot-devops-project/habot-devops-project/django/urls.py
12 would reformat /home/runner/work/habot-devops-project/habot-devops-project/django/views.py
13 would reformat /home/runner/work/habot-devops-project/habot-devops-project/django/madels.py
14 would reformat /home/runner/work/habot-devops-project/habot-devops-project/django/serializers.py
15
16 Oh no! ✖️❤️✖️
17 4 files would be reformatted.
18 Error: Process completed with exit code 1.
```

Run Flake8 0s

Test 2 – Flake8

- Add an unused import.
- **Expected:** Flake8 reports an error.

Summary

All jobs

security-checks

Run details

Usage

Workflow file

security-checks

failed now in 15s

Search logs

Install Python Tools

7s

Check Formatting

0s

Run Flake8

1s

Run Bandit Security Scan

0s

Scan for Secrets

0s

Setup Terraform

0s

Terraform Init

0s

Terraform Format Check

0s

Test 3 – Bandit

- Add insecure code (e.g., eval()).
- **Expected:** Bandit detects a security issue.

Summary

All jobs

security-checks

Run details

Usage

Workflow file

security-checks

failed now in 12s

Search logs

Run Bandit Security Scan

0s

```

1 ▶ Run bandit -r .
11 [main] INFO profile include tests: None
12 [main] INFO profile exclude tests: None
13 [main] INFO cli include tests: None
14 [main] INFO cli exclude tests: None
15 [main] INFO running on Python 3.12.13
16 Run started:2026-08-02 13:00:30.713869+00:00
17
18 Test results:
19 >> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer ast.literal_eval.
20 Severity: Medium Confidence: High
21 CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
22 More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist\_calls.html#b307-eval
23 Location: ./django/views.py:16:8
24 15 def test_bandit():
25 16     eval(input("Enter something: "))
26
27 -----
28
29 Code scanned:
30 Total lines of code: 45

```

Test 4 – Secret Scan

- Add a fake API key in a Python file.
- **Expected:** Secret scan detects the secret and fails.

Summary

All jobs

security-checks

Run details

Usage

Workflow file

security-checks

failed now in 16s

Search logs

Set up job

2s

Checkout Repository

0s

Setup Python

1s

Install Python Tools

8s

Check Formatting

1s

Run Flake8

1s

Run Bandit Security Scan

0s

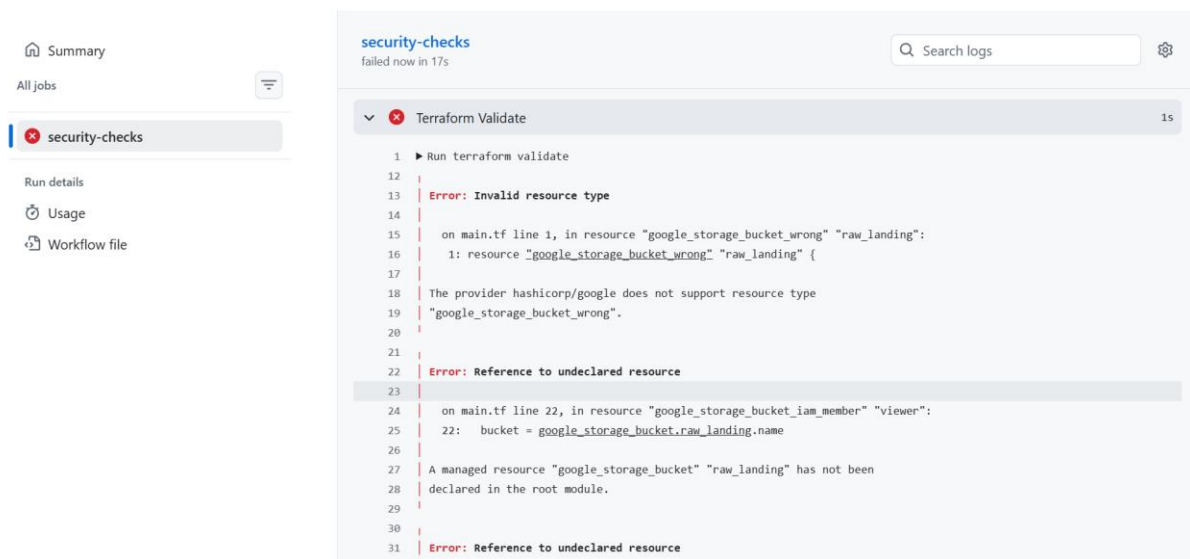
```

1 ▶ Run bandit -r .
11 [main] INFO profile include tests: None
12 [main] INFO profile exclude tests: None
13 [main] INFO cli include tests: None
14 [main] INFO cli exclude tests: None
15 [main] INFO running on Python 3.12.13
16 Run started:2026-08-02 13:22:23.234120+00:00
17 Test results:
18 >> Issue: [B105:hardcoded password string] Possible hardcoded password: 'mypassword123'

```

Test 5 – Terraform Validate

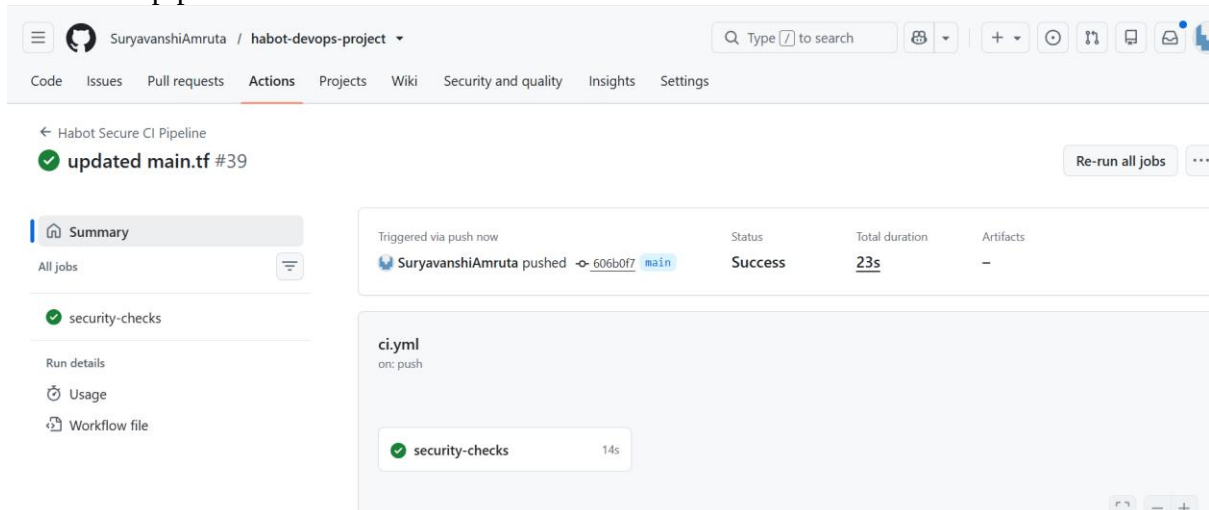
- Remove a closing brace (}) from a .tf file.
- **Expected:** Terraform validation fails.



The screenshot shows the GitHub Actions interface for a workflow named 'security-checks'. The left sidebar contains links for 'Summary', 'All jobs', 'Run details', 'Usage', and 'Workflow file'. The 'All jobs' section shows a failed job 'security-checks'. The main area displays the logs for the 'Terraform Validate' step, which failed with the following errors:

```
1  ▶ Run terraform validate
12
13  | Error: Invalid resource type
14  |
15  |   on main.tf line 1, in resource "google_storage_bucket_wrong" "raw_landing":
16  |     1: resource "google_storage_bucket_wrong" "raw_landing" {
17  |
18  |   The provider hashicorp/google does not support resource type
19  |   "google_storage_bucket_wrong".
20  |
21  |
22  | Error: Reference to undeclared resource
23  |
24  |   on main.tf line 22, in resource "google_storage_bucket_iam_member" "viewer":
25  |     22: bucket = google_storage_bucket.raw_landing.name
26  |
27  |   A managed resource "google_storage_bucket" "raw_landing" has not been
28  |   declared in the root module.
29  |
30  |
31  | Error: Reference to undeclared resource
```

Successful pipeline



The screenshot shows the GitHub Actions interface for a workflow named 'security-checks'. The left sidebar contains links for 'Summary', 'All jobs', 'Run details', 'Usage', and 'Workflow file'. The 'All jobs' section shows a successful job 'security-checks'. The main area displays the logs for the 'Terraform Validate' step, which passed successfully. The logs show the following output:

```
1  ▶ Run terraform validate
12
13  | Error: Invalid resource type
14  |
15  |   on main.tf line 1, in resource "google_storage_bucket_wrong" "raw_landing":
16  |     1: resource "google_storage_bucket_wrong" "raw_landing" {
17  |
18  |   The provider hashicorp/google does not support resource type
19  |   "google_storage_bucket_wrong".
20  |
21  |
22  | Error: Reference to undeclared resource
23  |
24  |   on main.tf line 22, in resource "google_storage_bucket_iam_member" "viewer":
25  |     22: bucket = google_storage_bucket.raw_landing.name
26  |
27  |   A managed resource "google_storage_bucket" "raw_landing" has not been
28  |   declared in the root module.
29  |
30  |
31  | Error: Reference to undeclared resource
```

Task 3 – Schema Mapping and DCYN Validation

1. Student Onboarding JSON

- Student details are submitted from the React application in JSON format.
- Contains student information such as name, age, email, phone, and learning difficulty.
- Sent to the Django REST API for validation.

Purpose: Standardize data exchange between frontend and backend.

Sample JSON

```
habot-devops-project > {} sample_request.json > abc email
1  {
2    "student_name": "Ishani Suryavanshi",
3    "age": 10,
4    "email": "amrutasuryavanshi212@gmail.com",
5    "guardian_name": "Amruta suryavanshi",
6    "phone": "9876543210",
7    "learning_difficulty": true
8  }
```

2. Why Schema Validation Matters

- Ensures all required fields are provided.
- Prevents invalid or incomplete data.
- Maintains data consistency.
- Protects database and analytics from incorrect records.
- Returns clear validation errors.

Purpose: Improve data quality and reliability.

3. Django REST Framework (DRF) Serializer

- Receives incoming JSON data.
- Validates each field.
- Rejects invalid requests.
- Saves only validated data to the database.

Purpose: Validate data before storage.

serializers.py

```
rs | main.tf | {} sample_request.json U | ! ci.yml | .gitignore | madels.py | serializers.py X
habot-devops-project > django > serializers.py
1  from rest_framework import serializers
2
3
4  class StudentSerializer(serializers.Serializer):
5
6      student_name = serializers.CharField(max_length=100)
7
8      age = serializers.IntegerField(min_value=3, max_value=18)
9
10     email = serializers.EmailField()
11
12     guardian_name = serializers.CharField(max_length=100)
13
14     phone = serializers.RegexField(regex=r"^\d{10}$")
15
16     learning_difficulty = serializers.BooleanField()
17
18     def validate_student_name(self, value):
19         if len(value.strip()) < 3:
20             raise serializers.ValidationError(
21                 "Student name must contain at least 3 characters."
22             )
23         return value
24
25     def validate_phone(self, value):
26         if not value.isdigit():
27             raise serializers.ValidationError("Phone number must contain digits only.")
28         return value
29
```

4. Field Validation

- Checks required fields.
- Verifies data type and format.
- Applies character and value limits.
- Ensures valid email and phone number.
- Validates Boolean fields.

Purpose: Ensure only valid data is accepted.

Field	Validation Rule	Accept	Reject
Student Name	Required, max 100 chars	✓	✗
Age	3–18	✓	✗
Email	Valid email	✓	✗
Phone	10 digits	✓	✗
Learning Difficulty	Boolean	✓	✗

5. DCYN (Yes/No) Logic

- Every field is checked against predefined rules.
- Validation has only two outcomes:
 - **Yes** → Accept
 - **No** → Reject
- Eliminates manual judgment.

Purpose: Apply consistent, rule-based validation.

Validation Check	Yes	No
Student Name provided	✓	✗
Age between 3–18	✓	✗
Valid Email	✓	✗
Phone has 10 digits	✓	✗
Boolean value	✓	✗

6. models.py

- Defines the Student database model.
- Specifies fields and data types.
- Creates the database table.

models.py

```
rs  main.tf  {} sample_request.json U  ! ci.yml  .gitignore  madels.py X  serializers.py

habot-devops-project > django > madels.py
1  from django.db import models
2
3
4  class Student(models.Model):
5      student_name = models.CharField(max_length=100)
6      age = models.PositiveIntegerField()
7      email = models.EmailField(unique=True)
8      guardian_name = models.CharField(max_length=100)
9      phone = models.CharField(max_length=10)
10     learning_difficulty = models.BooleanField(default=False)
11
12     def __str__(self):
13         return self.student_name
14
```

7. serializers.py

- Validates incoming JSON.
- Applies field validation rules.
- Returns validation errors.
- Saves validated records.

serializers.py

```
main.tf  {} sample_request.json  ! ci.yml  .gitignore  models.py  serializers.py  ×  
habot-devops-project > django > serializers.py  
1  from rest_framework import serializers  
2  
3  
4  class StudentSerializer(serializers.Serializer):  
5  
6      student_name = serializers.CharField(max_length=100)  
7  
8      age = serializers.IntegerField(min_value=3, max_value=18)  
9  
10     email = serializers.EmailField()  
11  
12     guardian_name = serializers.CharField(max_length=100)  
13  
14     phone = serializers.RegexField(regex=r"^\d{10}$")  
15  
16     learning_difficulty = serializers.BooleanField()  
17  
18     def validate_student_name(self, value):  
19         if len(value.strip()) < 3:  
20             raise serializers.ValidationError(  
21                 "Student name must contain at least 3 characters."  
22             )  
23         return value  
24  
25     def validate_phone(self, value):  
26         if not value.isdigit():  
27             raise serializers.ValidationError("Phone number must contain digits only.")  
28         return value  
29
```

8. views.py

- Handles API requests.
- Sends data to the serializer.
- Returns success or error responses.

views.py

```
habot-devops-project > django > views.py  
1  from rest_framework.response import Response  
2  from rest_framework.views import APIView  
3  
4  from .serializers import StudentSerializer  
5  
6  
7  class StudentAPIView(APIView):  
8  
9      def post(self, request):  
10  
11         serializer = StudentSerializer(data=request.data)  
12  
13         if serializer.is_valid():  
14  
15             serializer.save()  
16  
17             return Response(serializer.data, status=201)  
18  
19         return Response(serializer.errors, status=400)  
20
```

9. urls.py

- Defines API endpoints.
- Maps URLs to Django views.

urls.py

```
habot-devops-project > django > urls.py
1  from django.urls import path
2  from .views import StudentCreateView
3
4  urlpatterns = [
5      path("students/", StudentCreateView.as_view(), name="student-create"),
6  ]
7
```

10. Sample JSON

- Contains valid student onboarding data.
- Used to test the API.

Sample JSON Request

```
{
  "student_name": "Ishani Suryavanshi",
  "age": 10,
  "email": "amrutasuryavanshi212@gmail.com",
  "guardian_name": "Amruta suryavanshi",
  "phone": "9876543210",
  "learning_difficulty": true
}
```

Future Improvements

- Implement remote Terraform state management.
- Integrate Google Secret Manager for secure credential storage.
- Add monitoring and alerting using Google Cloud Monitoring.
- Extend the CI/CD pipeline with additional security and automated tests.
- Support multi-environment deployments (Development, Staging, Production).

Conclusion

This project demonstrates secure cloud infrastructure provisioning using **Terraform**, **Google Cloud Platform**, **GitHub Actions**, and **Django REST Framework**. It implements Infrastructure as Code (IaC), secure IAM practices, Row-Level Security (RLS), and an automated **Fail-Closed** CI/CD pipeline. The solution meets the project requirements by providing secure deployment, automated validation, and reliable data processing for the HabotConnect platform.

